Les Formulaires

Table des matières

La découpe 2
Générer le FormType 2
Adapter le Controller 3
Afficher le formulaire dans le template 4
Les Assertions 7

La découpe

Avec Symfony, un formulaire va être découpé en **trois** éléments :

- Une classe, décrivant le formulaire (ses champs, ses types etc. Très semblable à FormGroup dans Angular)
- Un Controller, qui va créer le formulaire, gérer sa soumission et gérer la persistance des données envoyées
- Un template, bien sûr ☺

Afin de nous aider à créer tout ça, nous allons avoir besoin d'installer le composant **Form**, et ses **Validators**, comme ceci :

```
composer require symfony/form
```

```
composer require symfony/validator
```

Générer le FormType

En se servant de l'Entity Auteur que Symfony connaît déjà, on va pouvoir lui faire générer un FormType en lui faisant reprendre toutes les propriétés de la classe Auteur. C'est tout l'intérêt de fonctionner avec le **MakerBundle** : Tout est standardisé, donc tout est facilement **chaînable**…

```
php bin/console make:form AuteurType
```

Et on va bien sûr lui donner le nom de l'entité sur laquelle il doit se baser : **Auteur**

```
The name of Entity or fully qualified model class name that the new form will be bound to (empty for none):
> Auteur
```

Allons voir le fichier généré :

```php
<?php

namespace App\Form;

use App\Entity\Auteur;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class AuteurType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('nom')
            ->add('prenom')
            ->add('nationalite')
            ->add('biographie')
            ->add('dateNaissance', null, [
                'widget' => 'single_text',
            ])
        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Auteur::class,
        ]);
    }
}
```

On peut ici rajouter un certain nombre d'éléments facultatifs mais qui vont nous faciliter la vie. Par exemple on peut rajouter un typage sur le champ Date pour bien le forcer à récuper une date_immutable (C'est le type qu'on lui a donné à la base). On peut également leur préciser un **Label** réutilisable par la suite. Et enfin on peut donner la Classe permettant de **Typer** la valeur qui sera récupérée dans ces inputs. Voici ma fonction buildForm après ces modifications :

```
public function buildForm(FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('nom', TextType::class, [
            'label' => 'Nom',
        ])
        ->add('prenom', TextType::class, [
            'label' => 'Prénom',
        ])
        ->add('nationalite', TextType::class, [
            'label' => 'Nationalité',
        ])
        ->add('dateNaissance', DateType::class, [
            'label'  => 'Date de naissance',
            'widget' => 'single_text',
            'input'  => 'datetime_immutable',
        ])
        ->add('biographie', TextareaType::class, [
            'label'    => 'Biographie',
            'required' => false,
        ]);
}
```

N'oubliez pas d'importer les classes TextType, DateType, TextareaType

Adapter le Controller

Dans notre AuteurController, on va vouloir préparer une nouvelle Route pour amener vers le form en question. On va appeler cette route /auteur/nouveau et on va la préparer comme ceci :

```
#[Route('/auteurs/nouveau', name: 'app_auteur_nouveau')]
public function nouveau(Request $request, EntityManagerInterface $em): Response
{
    $auteur = new Auteur();
    $form = $this->createForm(AuteurType::class, $auteur);

    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $em->persist($auteur);
        $em->flush();

        return $this->redirectToRoute('app_auteurs');
    }

    return $this->render('auteur/nouveau.html.twig', [
        'form' => $form,
    ]);
}
```

Attention : Vous allez avoir besoin d'importer quatre nouvelles classes en haut du fichier : la classe Auteur qui va nous servir à créer l'objet Auteur, AuteurType (le form que l'on vient de créer) et

EntityManagerInterface + la classe Request de HttpFoundation. L'auto-complétion devrait vous donner les chemins, sinon voici :

```
use App\Entity\Auteur;
use App\Form\AuteurType;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Component\HttpFoundation\Request;
```

Points importants à souligner :

handleRequest($request) analyse la requête HTTP — si c'est un GET, rien ne se passe. Si c'est un POST, il hydrate automatiquement l'objet **$auteur** avec les données soumises.

isSubmitted() && isValid() — les deux conditions sont indispensables. isSubmitted() vérifie qu'un formulaire a bien été envoyé, isValid() vérifie que les données passent les contraintes de validation.

EntityManagerInterface remplace ici le Repository pour la persistance — c'est l'interface principale de Doctrine pour écrire en base.

redirectToRoute() après un flush() réussi — c'est le pattern **Post/Redirect/Get**, qui évite la double soumission du formulaire si l'utilisateur recharge la page.

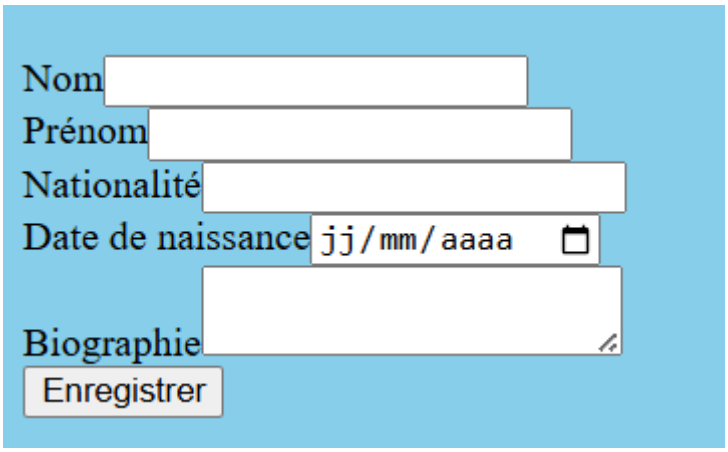
Afficher le formulaire dans le template

On va se créer un nouveau template dans un fichier qu'on va appeler **auteur/nouveau.html.twig**

Et on va y écrire ceci :

```
templates > auteur > nouveau.html.twig
1   {# templates/auteur/nouveau.html.twig #}
2   {% extends 'base.html.twig' %}
3
4   {% block title %}Nouvel Auteur{% endblock %}
5
6   {% block body %}
7       <h1>Ajouter un auteur</h1>
8
9       {{ form_start(form) }}
10          {{ form_row(form.nom) }}
11          {{ form_row(form.prenom) }}
12          {{ form_row(form.nationalite) }}
13          {{ form_row(form.dateNaissance) }}
14          {{ form_row(form.biographie) }}
15          <button type="submit">Enregistrer</button>
16      {{ form_end(form) }}
17  {% endblock %}
```

form_row est un raccourci de TWIG qui nous permet de générer directement la ligne du formulaire, label et input compris ☺ Regardons le résultat :



Ça fonctionne mais c'est moche.

Heureusement, TWIG permet de découper notre input en ses différentes parties, son label, son input etc…

Les fonctions Twig de rendu de formulaire :

Fonction	Rôle
form_start(form)	Génère la balise <form> avec l'action et la méthode
form_end(form)	Ferme la balise </form> et ajoute les champs cachés
form_widget(form.champ)	Affiche le champ de saisie uniquement
form_label(form.champ)	Affiche uniquement le label
form_error(form.champ)	Affiche les erreurs de validation du champ
form_row(form.champ)	Affiche label + widget + erreurs en une seule fois

On peut donc garder la main sur chaque élément séparément si on veut y appliquer du style. Mais surtout…

On peut dire à TWIG d'utiliser un thème Bootstrap ☺

Pour cela, rendez-vous dans /config/packages/twig.yaml :

```
twig:
    file_name_pattern: '*.twig'
    form_themes: ['bootstrap_5_layout.html.twig']

when@test:
    twig:
        strict_variables: true
```

Puis comme d'hab nous incluons les CDN Boostrap dans base.html.twig

Rechargeons notre page (je n'ai fait aucune modification dans le code de mon template) :

Les classes habituelles (form-control, form-label etc) se sont appliquées toutes seules ☺ Je vais maintenant adapter un peu tout ça et voici mon template final :

```
{# templates/auteur/nouveau.html.twig #}
{% extends 'base.html.twig' %}

{% block title %}Nouvel Auteur{% endblock %}

{% block body %}
    <div class="container py-5">
        <h1>Ajouter un auteur</h1>
        <div class="formulaire w-50">
            {{ form_start(form) }}
                <div class="row">
                    <div class="col-12 col-md-6">
                        {{ form_row(form.nom) }}
                    </div>
                    <div class="col-12 col-md-6">
                        {{ form_row(form.prenom) }}
                    </div>
                </div>
                <div class="row">
                    <div class="col-12 col-md-6">
                        {{ form_row(form.nationalite) }}
                    </div>
                    <div class="col-12 col-md-6">
                        {{ form_row(form.dateNaissance) }}
                    </div>
                </div>
                {{ form_row(form.biographie) }}
                <button type="submit" class="btn btn-primary mt-
3">Enregistrer</button>
            {{ form_end(form) }}
        </div>
    </div>
{% endblock %}
```

Les Assertions

Commençons par importer l'outil d'Assertion dans notre entité ☺

```
use Symfony\Component\Validator\Constraints as Assert;
```

Enfin, il ne nous reste plus qu'à mettre en place les messages d'erreur que l'on veut afficher si une contrainte n'est pas respectée. Pour cela, nous allons utiliser les annotations **Assert** directement dans l'entité Auteur. Prenons un exemple :

```
#[Assert\NotBlank]
private ?string $prenom = null;
```

Ici, on vérifie que le prénom n'est pas une chaîne vide par exemple. Mais ce n'est pas tout, on peut également lui préciser le message à afficher si cette erreur est rencontrée :

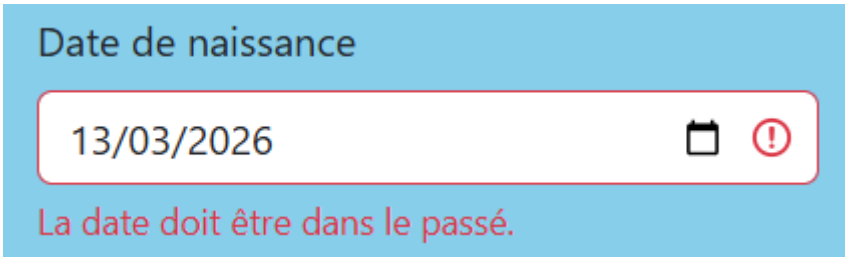
```
#[Assert\NotBlank(message: 'Le nom est obligatoire.')]
private ?string $nom = null;
```

On peut pousser les assertions assez loin, comme par exemple pour vérifier si la date de naissance est bien antérieure à la date du jour :

```
#[ORM\Column(type: Types::DATE_IMMUTABLE)]
#[Assert\NotNull(message: 'La date de naissance est obligatoire.')]
#[Assert\LessThan('today', message: 'La date doit être dans le passé.')]
private ?\DateTimeImmutable  $dateNaissance = null;
```

La liste de toutes les assertions est ici : https://symfony.com/doc/current/reference/constraints.html

Et lorsque l'on provoque une de ces erreurs, on a ceci dans notre formulaire :



Alors que le code du template, lui, ressemble toujours à ça :

```
{{ form_row(form.dateNaissance) }}
```

EXERCICE

1. Rajoutez des vérifications sur tous les champs de l'entité Auteur et essayez de faire afficher les message d'erreur.
2. Stylisez-moi tout ça et rendez-le joli parce qu'on va très vite le dupliquer ☺
3. Préparer un formulaire pour ajouter des livres en incluant un SELECT pour choisir quel est l'auteur du livre. Voici un guide étape par étape mais sans code : à vous de relire ce cours pour y arriver :

Étape 1 — Générer le FormType

Comme pour AuteurType, utilisez make:form pour générer LivreType lié à l'entité Livre.

Étape 2 — Le champ auteur dans le FormType

Dans le buildForm(), le champ auteur doit être de type EntityType — c'est le type Symfony prévu pour les relations. Il a besoin de trois options : class, choice_label, et placeholder. Cherchez EntityType dans la doc Symfony si besoin. Il y a un moyen d'afficher à la fois le nom et le prénom de l'auteur dans choice_label, je vous laisse chercher ☺

Étape 3 — La méthode nouveau() dans le contrôleur

Vous l'avez déjà faite pour AuteurController — la logique est **exactement identique**. Reproduisez-la dans LivreController en adaptant les noms.

Étape 4 — Le template

Même chose — reprenez votre auteur/nouveau.html.twig et adaptez-le pour les livres.

Étape 5 — Le lien vers le formulaire

Ajoutez un lien depuis votre page liste des livres.

Étape 6 — Mettez en place vos assertions et testez-les

N'oubliez pas de bien importer cette ligne dans votre entité :

```
use Symfony\Component\Validator\Constraint as Assert;
```